

COURSE CODE/TITLE:	CPE 212 Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	08/25/2026	DATE SUBMITTED:	08/25/2026
NAME:	Interno, Jerald San M.	INSTRUCTOR:	Engr. Jimlord

ACTIVITY NO. 4 Running Elevated Ad hoc Commands

1. PROCEDURE OUTPUT

What procedures did you perform?

Task 1: Run elevated ad hoc commands

1. Locally, we use the command `sudo apt update` when we want to download package information from all configured resources. The sources are often defined in `/etc/apt/sources.list` file and other files located in `/etc/apt/sources.list.d/` directory. So, when you run the update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run an apt update command in a remote machine. Issue the following command:

`ansible all -m apt -a update_cache=true`

```

gerald@workstation:~/ansible-lab$ ansible all -m apt -a update_cache=true -i inventory.yaml -k
SSH password:
server2 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)"
}
server1 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)"
}

```

What is the result of the command? Is it successful?

- **No, the command failed on both servers because updating the package cache requires root privileges (Permission denied).**

Try editing the command and add something that would elevate the privilege. Issue the command `ansible all -m apt -a update_cache=true --become --ask-become-pass`. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The `update_cache=true` is the same thing as running `sudo apt update`. The `--become` command elevates the privileges and the `--ask-become-pass` asks for the password. For now, even if we only changed the packaged index, we were able to change something on the remote server. You may notice after the second command was executed, the status is CHANGED compared to the first command, which is FAILED.

```
jerald@workstation:~/ansible-lab$ ansible all -m apt -a update_cache=true -i inventory.yaml -k --become --ask-become-pass
SSH password:
BECOME password[defaults to SSH password]:
server1 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787591139,
  "cache_updated": true,
  "changed": true
}
server2 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787591141,
  "cache_updated": true,
  "changed": true
}
```

2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: `ansible all -m apt -a name=vim-nox --become --ask-become-pass`. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.

```
jerald@workstation:~/ansible-lab$ ansible all -m apt -a name=vim-nox -i inventory.yaml -k --become --ask-become-pass
SSH password:
BECOME password[defaults to SSH password]:
server1 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787591139,
  "cache_updated": false,
  "changed": true,
  "stderr": "",
  "stderr_lines": [],
  "stdout": "Reading package lists...\nBuilding dependency tree...\nReading state information...\nThe following package was automatically installed and is no longer required:\n libfwupd2\nUse 'sudo apt autoremove' to remove it.\nThe following additional packages will be installed:\n fonts-lato javascript-common libjs-jquery liblua5.1-0 libruby libruby3.2\n libsodium23 rake ruby ruby-net-telnet ruby-rubygems ruby-sdbm ruby-webrick\n ruby-xmlrpc ruby3.2 rubygems-integration vim-common vim-runtime vim-tiny xxd\nSuggested packages:\n apache2 | lighttpd | httpd ri ruby-dev bundler cscope vim-doc indent\nThe following NEW packages will be installed:\n fonts-lato javascript-common libjs-jquery liblua5.1-0 libruby libruby3.2\n libsodium23 rake ruby ruby-net-telnet ruby-rubygems ruby-sdbm ruby-webrick\n ruby-xmlrpc ruby3.2 rubygems-integration vim-nox vim-runtime\nThe following packages will be upgraded:\n vim-common vim-tiny xxd\n3 upgraded, 18 newly installed, 0 to remove and 62 not upgraded.\nNeed to get 19.8 MB of archives.\nAfter this operation, 84.2 MB of additional disk space will be used.\nGet:1 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 fonts-lato all 2.015-1 [2781 kB]\nGet:2 http://ph.archive.ubuntu.com/ubuntu noble-updates/main amd64 vim-tiny amd64 2:9.1.0016-1ubuntu7.19 [806 kB]\nGet:3 http://ph.archive.ubuntu.com/ubuntu noble-updates/main amd64 vim-common all 2:9.1.0016-1ubuntu7.19 [388 kB]\nGet:4 http://ph.archive.ubuntu.com/ubuntu noble-updates/main amd64 xxd amd64 2:9.1.0016-1ubuntu7.19 [66.1 kB]\nGet:5 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 javascript-common all 11+nmu1 [5936 B]\nGet:6 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 libjs-jquery all 3.6.1+dfsg+~3.5.14-1 [328 kB]\nGet:7 http://ph.archive.ubuntu.com/ubuntu noble/universe amd64 liblua5.1-0 amd64 5.1.5-9build2 [120 kB]\nGet:8 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 libruby3.2 amd64 3.2.2-1ubuntu0.24.04.9 [50.7 kB]\nGet:9 http://ph.archive.ubuntu.com/ubuntu noble-updates/main amd64 ruby3.2 amd64 3.2.2-1ubuntu0.24.04.9 [50.7 kB]\nGet:10 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 rake all 13.0.1-4 [12.5 kB]\nGet:11 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 ruby-net-telnet all 0.0.1-1 [4476 B]\nGet:12 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 ruby-rubygems all 3.3.22-1 [258 kB]\nGet:13 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 ruby-sdbm all 1.0.5-1 [10.5 kB]\nGet:14 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 ruby-webrick all 1.8.10-1 [11.5 kB]\nGet:15 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 ruby-xmlrpc all 0.3.2-1 [10.5 kB]\nGet:16 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 vim-nox amd64 2:9.1.0016-1ubuntu7.19 [12.5 MB]\nGet:17 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 vim-runtime all 2:9.1.0016-1ubuntu7.19 [52.5 MB]\nGet:18 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 fonts-lato all 2.015-1 [2781 kB]\nFetched 19.8 MB in 1s (19.8 MB/s)\nSelecting previously unselected package fonts-lato.\n(Reading database ...)\nPreparing to unpack .../fonts-lato_2.015-1_all.deb ...\nUnpacking fonts-lato (2.015-1) ...\nSelecting previously unselected package javascript-common.\nPreparing to unpack .../javascript-common_11+nmu1_all.deb ...\nUnpacking javascript-common (11+nmu1) ...\nSelecting previously unselected package libjs-jquery.\nPreparing to unpack .../libjs-jquery_3.6.1+dfsg+~3.5.14-1_all.deb ...\nUnpacking libjs-jquery (3.6.1+dfsg+~3.5.14-1) ...\nSelecting previously unselected package liblua5.1-0.\nPreparing to unpack .../liblua5.1-0_5.1.5-9build2_amd64.deb ...\nUnpacking liblua5.1-0 (5.1.5-9build2) ...\nSelecting previously unselected package libruby3.2.\nPreparing to unpack .../libruby3.2_3.2.2-1ubuntu0.24.04.9_amd64.deb ...\nUnpacking libruby3.2 (3.2.2-1ubuntu0.24.04.9) ...\nSelecting previously unselected package ruby3.2.\nPreparing to unpack .../ruby3.2_3.2.2-1ubuntu0.24.04.9_amd64.deb ...\nUnpacking ruby3.2 (3.2.2-1ubuntu0.24.04.9) ...\nSelecting previously unselected package rake.\nPreparing to unpack .../rake_13.0.1-4_all.deb ...\nUnpacking rake (13.0.1-4) ...\nSelecting previously unselected package ruby-net-telnet.\nPreparing to unpack .../ruby-net-telnet_0.0.1-1_all.deb ...\nUnpacking ruby-net-telnet (0.0.1-1) ...\nSelecting previously unselected package ruby-rubygems.\nPreparing to unpack .../ruby-rubygems_3.3.22-1_all.deb ...\nUnpacking ruby-rubygems (3.3.22-1) ...\nSelecting previously unselected package ruby-sdbm.\nPreparing to unpack .../ruby-sdbm_1.0.5-1_all.deb ...\nUnpacking ruby-sdbm (1.0.5-1) ...\nSelecting previously unselected package ruby-webrick.\nPreparing to unpack .../ruby-webrick_1.8.10-1_all.deb ...\nUnpacking ruby-webrick (1.8.10-1) ...\nSelecting previously unselected package ruby-xmlrpc.\nPreparing to unpack .../ruby-xmlrpc_0.3.2-1_all.deb ...\nUnpacking ruby-xmlrpc (0.3.2-1) ...\nSelecting previously unselected package vim-common.\nPreparing to unpack .../vim-common_2:9.1.0016-1ubuntu7.19_all.deb ...\nUnpacking vim-common (2:9.1.0016-1ubuntu7.19) ...\nSelecting previously unselected package vim-tiny.\nPreparing to unpack .../vim-tiny_2:9.1.0016-1ubuntu7.19_amd64.deb ...\nUnpacking vim-tiny (2:9.1.0016-1ubuntu7.19) ...\nSelecting previously unselected package vim-nox.\nPreparing to unpack .../vim-nox_2:9.1.0016-1ubuntu7.19_amd64.deb ...\nUnpacking vim-nox (2:9.1.0016-1ubuntu7.19) ...\nSetting up fonts-lato (2.015-1) ...\nSetting up javascript-common (11+nmu1) ...\nSetting up libjs-jquery (3.6.1+dfsg+~3.5.14-1) ...\nSetting up liblua5.1-0 (5.1.5-9build2) ...\nSetting up libruby3.2 (3.2.2-1ubuntu0.24.04.9) ...\nSetting up ruby3.2 (3.2.2-1ubuntu0.24.04.9) ...\nSetting up rake (13.0.1-4) ...\nSetting up ruby-net-telnet (0.0.1-1) ...\nSetting up ruby-rubygems (3.3.22-1) ...\nSetting up ruby-sdbm (1.0.5-1) ...\nSetting up ruby-webrick (1.8.10-1) ...\nSetting up ruby-xmlrpc (0.3.2-1) ...\nSetting up vim-common (2:9.1.0016-1ubuntu7.19) ...\nSetting up vim-tiny (2:9.1.0016-1ubuntu7.19) ...\nSetting up vim-nox (2:9.1.0016-1ubuntu7.19) ...\nProcessing triggers for libc-bin (2.37-4ubuntu1) ..."
```

a. Verify that you have installed the package in the remote servers. Issue the command which vim and the command apt search vim-nox respectively.

```
jerald@workstation:~/ansible-lab$ ansible all -m command -a "which vim" -i inventory.yaml -k
SSH password:
server2 | CHANGED | rc=0 >>
/usr/bin/vim
server1 | CHANGED | rc=0 >>
/usr/bin/vim
```

```
jerald@workstation:~/ansible-lab$ ansible all -m command -a "apt search vim-nox" -i inventory.yaml -k
SSH password:
server1 | CHANGED | rc=0 >>
Sorting...
Full Text Search...
vim-nox/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.19 amd64 [installed]
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.19 amd64 [installed,automatic]
  Vi IMproved - enhanced vi editor - compact version
WARNING: apt does not have a stable CLI interface. Use with caution in scripts.
server2 | CHANGED | rc=0 >>
Sorting...
Full Text Search...
vim-nox/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.19 amd64 [installed]
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.19 amd64 [installed,automatic]
  Vi IMproved - enhanced vi editor - compact version
WARNING: apt does not have a stable CLI interface. Use with caution in scripts.
```

Was the command successful?

- Yes

b. Check the logs in the servers using the following commands: `cd /var/log`. After this, issue the command `ls`, go to the folder `apt` and open `history.log`.

```
jerald@workstation:~/ansible-lab$ ansible all -m command -a "cat /var/log/apt/history.log" -i inventory.yaml -k --become
--ask-become-pass
SSH password:
BECOME password[defaults to SSH password]:
server1 | CHANGED | rc=0 >>

Start-Date: 2026-08-25 01:10:07
Commandline: /usr/bin/apt-get -y -o Dpkg::Options::=--force-confdef -o Dpkg::Options::=--force-confold install vim-nox=2:9.1.0016-1ubuntu7.19
Requested-By: interno (1000)
Install: ruby-sdbm:amd64 (1.0.0-5build4, automatic), liblua5.1-0:amd64 (5.1.5-9build2, automatic), fonts-lato:amd64 (2.015-1, automatic), libruby:amd64 (1:3.2-ubuntu1, automatic), ruby-net-telnet:amd64 (0.2.0-1, automatic), rubygems-integration:amd64 (1.18, automatic), libruby3.2:amd64 (3.2.3-1ubuntu0.24.04.8, automatic), rake:amd64 (13.0.6-3, automatic), libsodium23:amd64 (1.0.18-1ubuntu0.24.04.1, automatic), vim-nox:amd64 (2:9.1.0016-1ubuntu7.19), ruby:amd64 (1:3.2-ubuntu1, automatic), vim-runtime:amd64 (2:9.1.0016-1ubuntu7.19, automatic), ruby3.2:amd64 (3.2.3-1ubuntu0.24.04.8, automatic), libjs-jquery:amd64 (3.6.1+dfsg+~3.5.14-1, automatic), ruby-rubygems:amd64 (3.4.20-1, automatic), javascript-common:amd64 (11+nmu1, automatic), ruby-xmlrpc:amd64 (0.3.2-2, automatic), ruby-webrick:amd64 (1.8.1-1ubuntu0.2, automatic)
Upgrade: xxd:amd64 (2:9.1.0016-1ubuntu7.18, 2:9.1.0016-1ubuntu7.19), vim-common:amd64 (2:9.1.0016-1ubuntu7.18, 2:9.1.0016-1ubuntu7.19), vim-tiny:amd64 (2:9.1.0016-1ubuntu7.18, 2:9.1.0016-1ubuntu7.19)
End-Date: 2026-08-25 01:10:41
server2 | CHANGED | rc=0 >>

Start-Date: 2025-08-05 16:48:47
Commandline: apt-get -y --fix-policy install
Install: libgpg-error-l10n:amd64 (1.47-3build2, automatic), e2fsprogs-l10n:amd64 (1.47.0-2.4-exp1ubuntu4, automatic), libgpm2:amd64 (1.20.7-11, automatic), psmisc:amd64 (23.7-1build1, automatic), uuid-runtime:amd64 (2.39.3-9ubuntu6, automatic), bash-completion:amd64 (1:2.11-8, automatic), bsdxtractils:amd64 (2.39.3-9ubuntu6, automatic)
End-Date: 2025-08-05 16:48:49

Start-Date: 2025-08-05 16:49:12
Commandline: apt-get --yes -oDebug::pkgDepCache::AutoInstall=yes --force-yes upgrade
Upgrade: libao-error-l10n:amd64 (1.47-3build2, 1.47-3build2.1), doko:amd64 (1.22.6ubuntu6, 1.22.6ubuntu6.1), libsm
```

Describe what you see in the `history.log`.

- The `history.log` file records the timestamp, requesting user, executed APT command (apt install vim-nox), and a list of installed dependencies

3. This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

a. Issue the command: `ansible all -m apt -a name=snapd --become --ask-become-pass`

```
jerald@workstation:~/ansible-lab$ ansible all -m apt -a name=snapd -i inventory.yaml -k --become --ask-become-pass
SSH password:
BECOME password[defaults to SSH password]:
server1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787591139,
  "cache_updated": false,
  "changed": false
}
server2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787591141,
  "cache_updated": false,
  "changed": false
}
```

Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?

- The output displays **SUCCESS** with **changed: false**. Nothing changes because snapd is already pre-installed on standard Ubuntu releases.

b. Now, try to issue this command: `ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass`

```
jerald@workstation:~/ansible-lab$ ansible all -m apt -a "name=snapd state=latest" -i inventory.yaml -k --become --ask-become-pass
SSH password:
BECOME password[defaults to SSH password]:
server2 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787591141,
  "cache_updated": false,
  "changed": true,
  "stderr": "",
  "stderr_lines": [],
  "stdout": "Reading package lists...\nBuilding dependency tree...\nReading state information...\nThe following package was automatically installed and is no longer required:\n libfwupd2\nUse 'sudo apt autoremove' to remove it.\nThe following packages will be upgraded:\n snapd\n1 upgraded, 0 newly installed, 0 to remove and 61 not upgraded.\nNeed to get 36.6 MB of archives.\nAfter this operation, 651 kB of additional disk space will be used.\nGet:1 http://ph.archive.ubuntu.com/ubuntu noble-updates/main amd64 snapd amd64 2.76.3+ubuntu24.04 [36.6 MB]\nFetch 36.6 MB in 18s (2068 kB/s)\n(Reading database ... \r(Reading database ... 5%\r(Reading database ... 10%\r(Reading database ... 15%\r(Reading database ... 20%\r(Reading database ... 25%\r(Reading database ... 30%\r(Reading database ... 35%\r(Reading database ... 40%\r(Reading database ... 45%\r(Reading database ... 50%\r(Reading database ... 55%\r(Reading database ... 60%\r(Reading database ... 65%\r(Reading database ... 70%\r(Reading database ... 75%\r(Reading database ... 80%\r(Reading database ... 85%\r(Reading database ... 90%\r(Reading database ... 95%\r(Reading database ... 100%\r(Reading database ... 161267 files and directories currently installed.)\nPreparing to unpack .../snapd_2.76.3+ubuntu24.04_amd64.deb ...\nUnpacking snapd (2.76.3+ubuntu24.04) over (2.76+ubuntu24.04.1) ...\nSetting up snapd (2.76.3+ubuntu24.04) ...\nnsnapd.failure.service is a disabled or a static unit not running, not starting it.\nnsnapd.gpio-chardev-setup.target is a disabled or a static unit not running, not starting it.\nnsnapd.snap-repair.service is a disabled or a static unit not running, not starting it.\nProcessing triggers for gnome-menus (3.36.0-1.1ubuntu3) ...\nProcessing triggers for man-db (2.12.0-4build2) ...\nProcessing triggers for dbus (1.14.10-4ubuntu4.1) ...\nProcessing triggers for desktop-file-utils (0.27-2build1) ...\n\n...\n"
  "stdout_lines": [

```

Describe the output of this command. Notice how we added the command `state=latest` and placed them in double quotations.

- As I observed, adding `state=latest` forces Ansible to query the remote repository for updates. It returns `ok (changed: false)` if `snappd` is already up to date, or `CHANGED (changed: true)` if an upgrade was applied.

Task 2: Writing our First Playbook

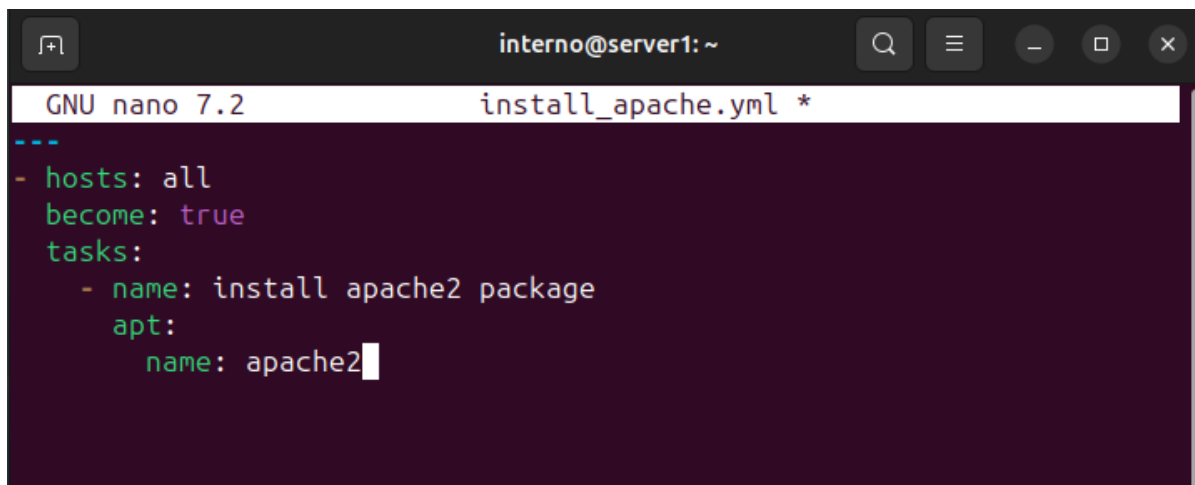
1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (CPE232_yourname). Issue the command `nano install_apache.yml`. This will create a playbook file called `install_apache.yml`. The `.yml` is the basic standard extension for playbook files.

When the editor appears, type the following:

Make sure to save the file.

Take note also of the alignments of the texts. Use TAB instead of SPACE BAR for spacing.

```
jerald@workstation:~/ansible-lab$ nano install_apache.yml
```



```

interno@server1: ~
GNU nano 7.2 install_apache.yml *
---
- hosts: all
  become: true
  tasks:
    - name: install apache2 package
      apt:
        name: apache2
  
```

2. Run the yml file using the command: `ansible-playbook --ask-become-pass install_apache.yml`.

```
jerald@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml install_apache.yml -k --ask-become-pass
SSH password:
BECOME password[defaults to SSH password]:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server2]
ok: [server1]

TASK [install apache2 package] *****
changed: [server1]
changed: [server2]

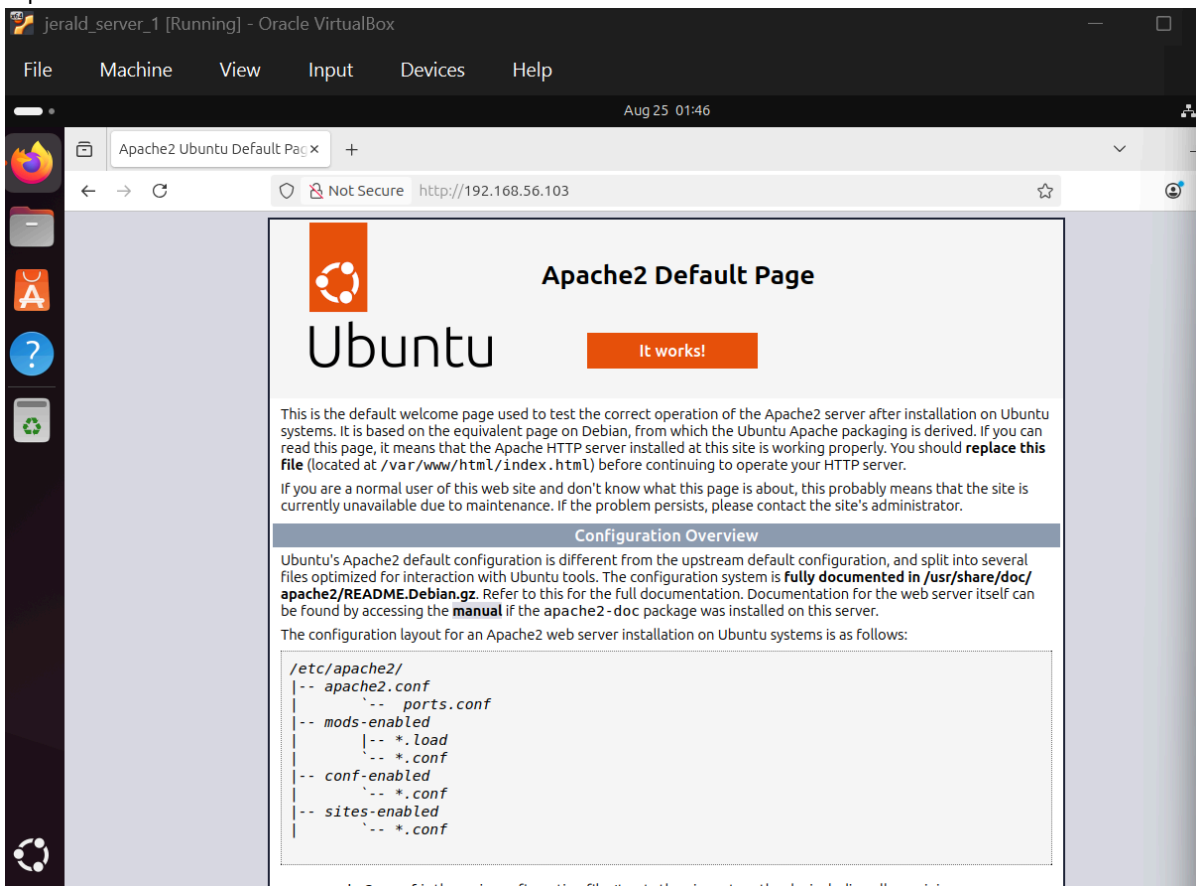
PLAY RECAP *****
server1                : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server2                : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Describe the result of this command.

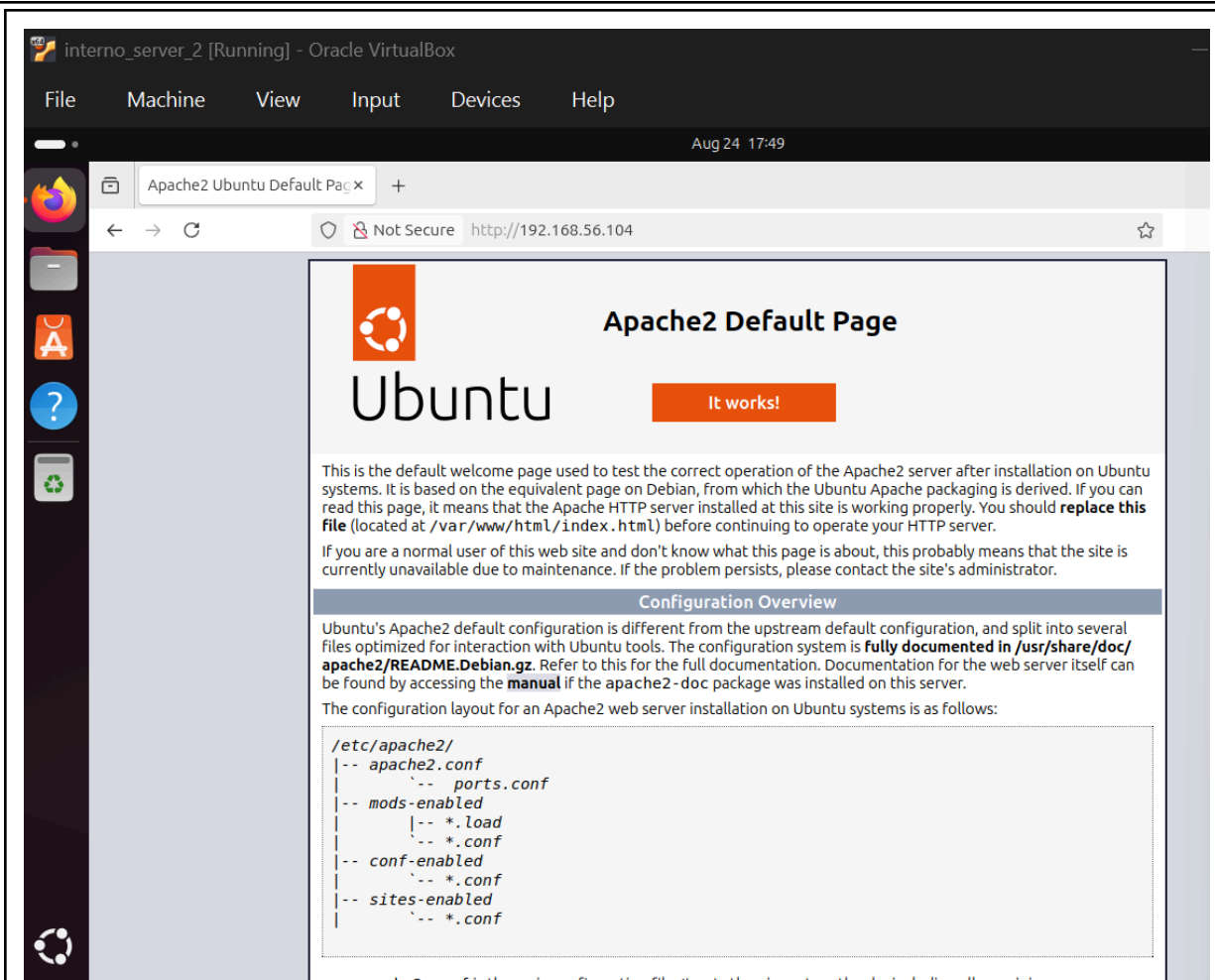
- The ansible connects to server1 and server2, installs apache2, and outputs changed=1 in yellow text for both hosts, signaling successful package installation.

3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.

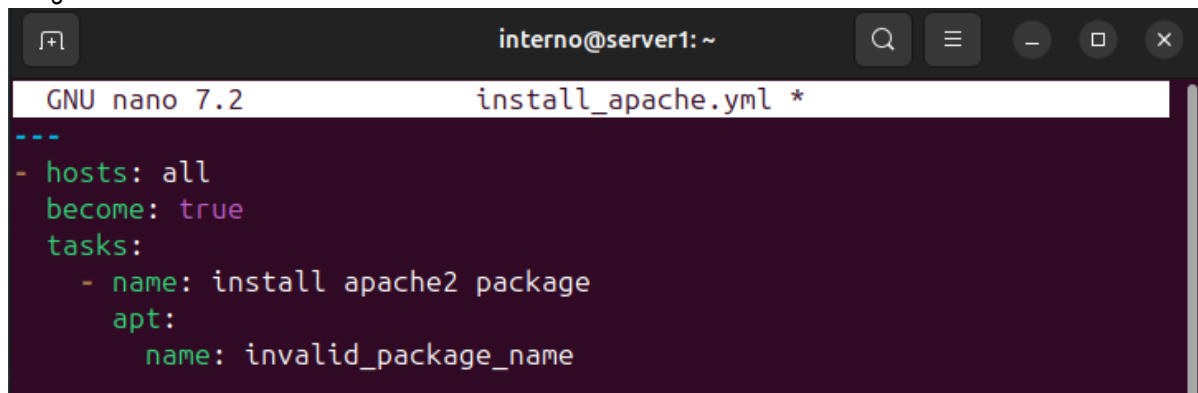
http:192.168.56.103



http:192.168.56.104



4. Try to edit the `install_apache.yml` and change the name of the package to any name that will not be recognized.



```
jerald@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml install_apache.yml -k --ask-become-pass
SSH password:
BECOME password[defaults to SSH password]:

PLAY [all] *****

TASK [Gathering Facts] *****
fatal: [server1]: UNREACHABLE! => {"changed": false, "msg": "Failed to connect to the host via ssh: ssh: connect to host 192.168.56.103 port 22: Connection timed out", "unreachable": true}
ok: [server2]

TASK [install apache2 package] *****
fatal: [server2]: FAILED! => {"changed": false, "msg": "No package matching 'invalid_package_name' is available"}

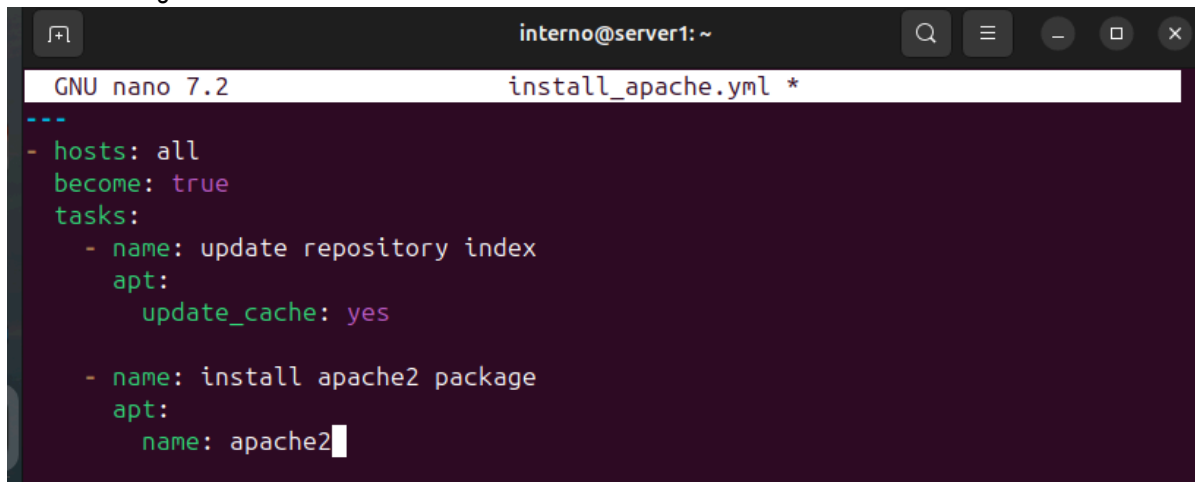
PLAY RECAP *****
server1          : ok=0    changed=0    unreachable=1    failed=0    skipped=0    rescued=0
ignored=0
server2          : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
ignored=0
```

What is the output?

- The task fails with a fatal: FAILED! error message in red text because APT cannot find a package named invalid_package_name in the Ubuntu repository index.

5. This time, we are going to put additional tasks into our playbook. Edit the install_apache.yml. As you can see, we are now adding an additional command, which is the update_cache. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.

Save the changes to this file and exit.



```
GNU nano 7.2 install_apache.yml *
---
- hosts: all
  become: true
  tasks:
    - name: update repository index
      apt:
        update_cache: yes
    - name: install apache2 package
      apt:
        name: apache2
```

6. Run the playbook and describe the output.

```
jerald@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml install_apache.yml -k --ask-become-pass
SSH password:
BECOME password[defaults to SSH password]:

PLAY [all] *****
*

TASK [Gathering Facts] *****
*
ok: [server2]
fatal: [server1]: UNREACHABLE! => {"changed": false, "msg": "Failed to connect to the host via ssh: ssh: connect to host 192.168.56.103 port 22: Connection timed out", "unreachable": true}

TASK [update repository index] *****
*
changed: [server2]

TASK [install apache2 package] *****
*
ok: [server2]

PLAY RECAP *****
*
server1      : ok=0    changed=0    unreachable=1    failed=0
skipped=0    rescued=0    ignored=0
server2      : ok=3    changed=1    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
```

Did the new command change anything on the remote servers?

- **Yes. The `update_cache: yes` module task modified the remote servers by refreshing their APT package index files stored in `/var/lib/apt/lists/`. This ensures `server1` and `server2` have the most up-to-date package lists from the official Ubuntu software repositories. However, it did not install, upgrade, or alter any actual installed software packages on the remote systems.**

7. Edit again the `install_apache.yml`. This time, we are going to add a PHP support for the apache package we installed earlier.

```

GNU nano 7.2                                interno@server1: ~
--
- hosts: all
  become: true
  tasks:
    - name: update repository index
      apt:
        update_cache: yes

    - name: install apache2 package
      apt:
        name: apache2

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
  
```

Save the changes to this file and exit.

8. Run the playbook and describe the output.

```

jerald@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml install_apache.yml -k
--ask-become-pass
SSH password:
BECOME password[defaults to SSH password]:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]

TASK [update repository index] *****
changed: [server2]
changed: [server1]

TASK [install apache2 package] *****
ok: [server1]
ok: [server2]

TASK [add PHP support for apache] *****
changed: [server1]
changed: [server2]

PLAY RECAP *****
server1                : ok=4    changed=2    unreachable=0    failed=0    skipped=0
  rescued=0    ignored=0
server2                : ok=4    changed=2    unreachable=0    failed=0    skipped=0
  rescued=0    ignored=0
  
```

Did the new command change anything on the remote servers?

- **Yes, the new command changed both remote servers by downloading and installing the libapache2-mod-php package alongside its required dependencies. As shown in your execution output, TASK [add PHP support for apache] returned changed, successfully configuring both server1 and server2 with PHP support for the Apache web server.**

9. Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository.

```
jerald@workstation:~/ansible-lab$ git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 6 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.15 KiB | 587.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/JeraldSanInterno/CPE232_Jerald_Interno.git
 8b5a88b..5a12929  main -> main
branch 'main' set up to track 'origin/main'.
jerald@workstation:~/ansible-lab$
```

Link: https://github.com/JeraldSanInterno/CPE232_Jerald_Interno.git

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Answer the following:

1. What is the importance of using a playbook?

- **A playbook automates repetitive tasks, ensuring that configurations are applied quickly, consistently, and without human error across multiple servers.**

2. Summarize what we have done on this activity.

- **In this activity, we created and executed an Ansible playbook (install_apache.yml) to automatically update package caches, install Apache2, and add PHP support across managed servers (server1 and server2). After verifying that the web server was successfully deployed, we initialized a local Git repository, authenticated with GitHub using a Personal Access Token (PAT), and pushed our Ansible code to a remote repository.**

Conclusions/Learnings:

- **Through this activity, we learned how Ansible playbooks simplify system administration by automating identical software installations across multiple servers efficiently and without human error. We also gained practical experience managing infrastructure code with Git, understanding how to handle remote merge conflicts and securely authenticate pushes to GitHub using Personal Access Tokens (PAT).**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT



I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: idea generation and code debugging.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.